

Roteiro de Laboratório: Metadados de Governança e Problemas em ITIL com Python

Curso: Sistemas de Informação

Disciplina: Gestão de Sistemas de Informação

Tema: Governança de TI – ITIL (Incidentes, Problemas, SLA e Metadados)

1. Fundamentação Teórica (resumo)

Na Aula 7, discutimos:

- **Incidente**: interrupção ou degradação de um serviço (sintoma).
- **Problema**: causa raiz desconhecida de um ou mais incidentes (causa).
- **Gerenciamento de Incidentes**: restaurar a operação normal o mais rápido possível, minimizando o impacto no negócio.
- **Gerenciamento de Problemas**: investigar causas raiz e evitar recorrência.

Vimos também o papel dos **metadados de governança**:

- **Prioridade** = impacto no negócio + urgência (define o que é atendido primeiro).
- **Severidade** = gravidade técnica (quão “quebrado” o serviço está).

Por fim, revisamos o conceito de **SLA (Service Level Agreement)**:

- Acordo formal que define metas mensuráveis (ex.: “Prioridade Alta deve ser resolvida em até 2 horas”).
- Base para **governança baseada em evidência**, via comparação entre **meta em política** e **dados reais** (teste substantivo).

Neste laboratório, você irá:

1. Usar **Prioridade** e **Severidade** para triagem e escalonamento de incidentes.
2. Identificar **incidentes recorrentes** e abrir **Problemas** automaticamente.
3. Gerar um **relatório de governança** que una Incidentes, Problemas e cumprimento de SLA.

As atividades seguem usando a empresa fictícia **TechStore S.A.**.

2. Laboratório Prático – Metadados, Incidentes e Problemas

Atividade 1 – Triagem com Prioridade e Severidade

Objetivo:

Simular uma fila de incidentes da TechStore S.A., usando **Prioridade** e **Severidade** para:

- classificar os chamados;
- determinar o **nível de escalonamento** (N1 ou N2);
- marcar possíveis violações de SLA para prioridade Alta.

Cenário:

A TechStore S.A. definiu em seu SLA:

- Incidentes de **Prioridade Alta** devem ser resolvidos em até **2 horas**.

Além da Prioridade, o Service Desk passou a registrar também a **Severidade** técnica:

- Severidade 1: Crítico
- Severidade 2: Alto
- Severidade 3: Médio
- Severidade 4: Baixo

Regras simplificadas de triagem:

- Se **Prioridade = Alta** e **Severidade em {1,2}** → escalonar para **Nível 2 (N2)**.
- Caso contrário → manter em **Nível 1 (N1)**.

Use o código-base abaixo:

```
import pandas as pd
import random

# =====
# ATIVIDADE 1 — TRIAGEM COM PRIORIDADE E SEVERIDADE
# =====

random.seed(42)

# 1. Geração de log fictício de incidentes
n_incidentes = 25
prioridades = ['Alta', 'Media', 'Baixa']
severidades = [1, 2, 3, 4] # 1 = mais grave, 4 = menos grave

dados_incidentes = {
    'ticket_id': range(4001, 4001 + n_incidentes),
    'prioridade': [random.choice(prioridades) for _ in range(n_incidentes)],
    'severidade': [random.choice(severidades) for _ in range(n_incidentes)],
    # tempo de resolução entre 0.5h e 5h
}
```

```

        'tempo_resolucao_horas': [round(random.uniform(0.5, 5.0), 2) for _ in
range(n_incidentes)]
    }

df_triagem = pd.DataFrame(dados_incidentes)

# Meta de SLA para prioridade Alta
meta_sla_alta = 2.0 # horas

# --- INÍCIO DA TAREFA DO ALUNO ---

# 2. Criar uma nova coluna 'fila_destino' com base em Prioridade e Severidade
# Regra:
# - Se prioridade = 'Alta' e severidade em {1,2} -> 'N2'
# - Caso contrário -> 'N1'

def decidir_fila(prioridade, severidade):
    # Substitua o 'pass' pela lógica condicional descrita acima
    pass

df_triagem['fila_destino'] = df_triagem.apply(
    lambda linha: decidir_fila(linha['prioridade'], linha['severidade']),
    axis=1
)

# 3. Identificar violações de SLA para chamados de Prioridade Alta
# Critério: tempo_resolucao_horas > meta_sla_alta

violacoes_sla_alta = df_triagem[
    (df_triagem['prioridade'] == 'Alta') &
    (df_triagem['tempo_resolucao_horas'] > meta_sla_alta)
]

# 4. Exibir relatório de triagem e violações

print("=== LOG DE INCIDENTES - TRIAGEM POR PRIORIDADE E SEVERIDADE (TECHSTORE S.A.)  

===")
print(df_triagem)

print("\n" + "=" * 70)
print("RELATÓRIO DE GOVERNANÇA - VIOLAÇÕES DE SLA (PRIORIDADE ALTA)")
print(f"Critério: tempo_resolucao_horas > {meta_sla_alta} horas")

```

```

print("=" * 70)

if violacoes_sla_alta.empty:
    print("Nenhuma violação de SLA para incidentes de prioridade Alta.")
else:
    print(violacoes_sla_alta)
    print(f"\nTotal de violações de SLA (Alta): {len(violacoes_sla_alta)}")

# --- FIM DA TAREFA DO ALUNO ---

```

Atividade 2 – Identificação de Problemas a partir de Incidentes Recorrentes

Objetivo:

Usar o log de incidentes da Atividade 1 para:

- identificar **incidentes recorrentes** de um mesmo tipo/serviço;
- abrir **Registros de Problema** automaticamente quando a recorrência ultrapassar um limite.

Cenário simplificado:

Cada incidente está associado a um **serviço**:

- ERP, Portal do Cliente, Sistema de Pagamentos, E-mail Corporativo.

A política da TechStore S.A. definiu:

- Se um mesmo serviço tiver **3 ou mais incidentes de Prioridade Alta** no período, deve ser **aberto um Problema** para investigação da causa raiz.

Adapte o código abaixo (você pode rodar na mesma célula ou em uma nova, reaproveitando o df_triagem):

```

import pandas as pd
import random

# =====
# ATIVIDADE 2 — IDENTIFICAÇÃO DE PROBLEMAS (ITIL)
# =====

random.seed(99)

# 1. Partindo de um novo log de incidentes com serviço associado
n_incidentes = 30
prioridades = ['Alta', 'Media', 'Baixa']

```

```

severidades = [1, 2, 3, 4]
servicos = ['ERP', 'Portal do Cliente', 'Sistema de Pagamentos', 'E-mail Corporativo']

dados_incidentes2 = {
    'ticket_id': range(5001, 5001 + n_incidentes),
    'servico': [random.choice(servicos) for _ in range(n_incidentes)],
    'prioridade': [random.choice(prioridades) for _ in range(n_incidentes)],
    'severidade': [random.choice(severidades) for _ in range(n_incidentes)],
}

df_incidentes = pd.DataFrame(dados_incidentes2)

# --- INÍCIO DA TAREFA DO ALUNO ---

print("=== LOG DE INCIDENTES POR SERVIÇO — TECHSTORE S.A. ===")
print(df_incidentes)

# 2. Filtrar apenas incidentes de Prioridade Alta
incidentes_alta = df_incidentes[df_incidentes['prioridade'] == 'Alta']

# 3. Contar quantos incidentes de Prioridade Alta houve por serviço
contagem_alta_por_servico = incidentes_alta.groupby('servico')['ticket_id'].count()

print("\n=== CONTAGEM DE INCIDENTES DE PRIORIDADE ALTA POR SERVIÇO ===")
print(contagem_alta_por_servico)

# 4. Definir threshold para abrir Problema (por exemplo, 3 incidentes Alta)
limite_problema = 3

# 5. Criar uma tabela de "Problemas" para serviços que atingirem o limite
# Dica: selecione apenas serviços com contagem >= limite_problema

servicos_com_problema = contagem_alta_por_servico[contagem_alta_por_servico >=
limite_problema]

df_problemas = pd.DataFrame({
    'servico': servicos_com_problema.index,
    'qtd_incidentes_alta': servicos_com_problema.values,
    'status_problema': ['Aberto'] * len(servicos_com_problema)
})

print("\n" + "=" * 70)

```

```

print("REGISTROS DE PROBLEMAS ABERTOS (SERVIÇOS COM INCIDENTES ALTA
RECORRENTES)")
print("=" * 70)

if df_problemas.empty:
    print("Nenhum Problema aberto. Nenhum serviço atingiu o limite de recorrência.")
else:
    print(df_problemas)

# --- FIM DA TAREFA DO ALUNO ---

```

Atividade 3 – Painel de Governança: Incidentes, Problemas e SLA

Objetivo:

Unir em um único relatório:

- o volume total de incidentes por serviço;
- a quantidade de incidentes de Prioridade Alta;
- se há Problema aberto para o serviço;
- a taxa de serviços com Problemas em relação ao total (indicador de risco).

Cenário:

Reaproveite os dados da **Atividade 2**:

- df_incidentes (log de incidentes com serviço, prioridade, severidade);
- df_problemas (serviços que ultrapassaram o limite de incidentes Alta).

Monte um painel de governança conforme o código-base:

```

import pandas as pd

# =====
# ATIVIDADE 3 — PAINEL DE GOVERNANÇA (RESUMO)
# =====

# Supõe que df_incidentes e df_problemas já existam na memória
# Se não existirem, rode antes a Atividade 2.

# --- INÍCIO DA TAREFA DO ALUNO ---

# 1. Contagem total de incidentes por serviço

```

```

total_incidentes_por_servico = df_incidentes.groupby('servico')['ticket_id'].count()

# 2. Contagem de incidentes de Prioridade Alta por serviço
incidentes_alta = df_incidentes[df_incidentes['prioridade'] == 'Alta']
incidentes_alta_por_servico = incidentes_alta.groupby('servico')['ticket_id'].count()

# 3. Montar um DataFrame de painel unificando essas informações
painel = pd.DataFrame({
    'total_incidentes': total_incidentes_por_servico,
    'incidentes_prioridade_alta': incidentes_alta_por_servico
})

# Preencher com 0 onde não houver incidentes Alta
painel['incidentes_prioridade_alta'] = painel['incidentes_prioridade_alta'].fillna(0).astype(int)

# 4. Adicionar coluna indicando se há Problema aberto para o serviço
painel['problema_aberto'] = painel.index.isin(df_problemas['servico']).astype(int)

print("=== PAINEL DE GOVERNANÇA — INCIDENTES, PRIORIDADE ALTA E PROBLEMAS ===")
print(painel)

# 5. Calcular taxa de serviços com Problema aberto
qtd_servicos = len(painel)
qtd_com_problema = painel['problema_aberto'].sum()

taxa_servicos_com_problema = round((qtd_com_problema / qtd_servicos) * 100, 1)

print("\nResumo:")
print(f"Total de serviços monitorados: {qtd_servicos}")
print(f"Serviços com Problema aberto: {qtd_com_problema}")
print(f"Taxa de serviços com Problema aberto: {taxa_servicos_com_problema}%")

# --- FIM DA TAREFA DO ALUNO ---

```